BASIC ELECTRONICS ENGINEERING • PART B

UNIT III

Digital Electronics

Number systems, codes, logic gates, Boolean algebra, adders, flip-flops, registers and counters

WHAT YOU WILL LEARN

1 Number systems & conversions
2 BCD, Excess-3, Gray & Hamming codes
3 Logic gates & universal gates
4 Boolean algebra & theorems
5 Half adder & full adder
6 Sequential circuits & flip-flops
7 Registers and counters
8 Summary & exam questions

1 Overview of Number Systems

Digital circuits work with only two voltage levels, **HIGH (1)** and **LOW (0)**, so they naturally use the **binary** number system. Octal and hexadecimal are shorthand ways to write long binary numbers.

System	Base	Digits used	Example
Decimal	10	0–9	$(45)_{10}$
Binary	2	0, 1	$(101101)_{2}$
Octal	8	0–7	$(55)_{8}$
Hexadecimal	16	0–9, A–F (A=10 … F=15)	$(2D)_{16}$

Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Oct	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

Conversion methods

Conversion	Method	Example
Decimal → Binary	Divide repeatedly by 2; read remainders **bottom to top**	45 → 22 r1, 11 r0, 5 r1, 2 r1, 1 r0, 0 r1 → **101101**
Binary → Decimal	Add the weights (…8, 4, 2, 1 . ½, ¼, ⅛)	1101.101 = 8+4+1+0.5+0.125 = **13.625**
Binary → Octal	Group bits in **3s** from the binary point	101 110 011 → $\mathbf{563}_{8}$
Binary → Hex	Group bits in **4s**	1011 1100 → $\mathbf{BC}_{16}$
Hex → Decimal	Multiply digits by powers of 16	2F = 2×16 + 15 = **47**
Decimal → Hex	Divide repeatedly by 16	255 → F, F → **FF**

Binary arithmetic

Addition: 0+0 = 0, 0+1 = 1, 1+0 = 1, **1+1 = 10** (sum 0, carry 1). **Subtraction** is done by adding the **2's complement**.

1's complement = flip every bit; **2's complement** = 1's complement + 1.

Example (4-bit) 7 − 5: 0111 + 2's comp(0101) = 0111 + 1011 = 1 0010 → discard the carry → **0010 = 2** ✓

2 Digital Codes: BCD, Excess-3, Gray and Hamming

A **code** represents numbers or characters as groups of bits. Different codes are chosen for ease of calculation, error reduction or error correction.

Decimal	Binary	BCD (8421)	Excess-3	Gray
0	0000	0000	0011	0000
1	0001	0001	0100	0001
2	0010	0010	0101	0011
3	0011	0011	0110	0010
4	0100	0100	0111	0110
5	0101	0101	1000	0111
6	0110	0110	1001	0101
7	0111	0111	1010	0100
8	1000	1000	1011	1100
9	1001	1001	1100	1101

BCD (Binary Coded Decimal)

Each decimal **digit** is written separately as a 4-bit binary number (weights 8-4-2-1). Codes 1010–1111 are **invalid**.

Example: $(47)_{10}$ = **0100 0111**$_{BCD}$. Used in digital clocks, calculators and meters.

Excess-3 code

BCD + 0011 (3) for every digit. It is **self-complementing** (9's complement = flip all bits), useful in decimal arithmetic.

Example: 5 → 0101 + 0011 = **1000**.

Gray code (reflected binary)

Only one bit changes between consecutive numbers, which avoids glitches in encoders and position sensors.

Binary → Gray: keep the MSB; each next Gray bit = XOR of that binary bit and the one before it. 1011 → 1, 1⊕0=1, 0⊕1=1, 1⊕1=0 → **1110**.

Gray → Binary: keep the MSB; each next binary bit = previous binary bit ⊕ current Gray bit. 1110 → 1, 1⊕1=0, 0⊕1=1, 1⊕0=1 → **1011**.

Hamming code (error detection and correction)

- A **Hamming (7,4)** code adds **3 parity bits** to 4 data bits so that a **single-bit error** can be found *and corrected*.
- Bit positions 1 to 7: parity bits sit at positions **1, 2, 4** (powers of 2); data bits at 3, 5, 6, 7.
- Each parity bit checks a group (even parity): **P1** → positions 1,3,5,7; **P2** → 2,3,6,7; **P4** → 4,5,6,7.

Step	Working for data 1011 (even parity)
Place data	Positions 3,5,6,7 = **1, 0, 1, 1**
P1 (1,3,5,7)	bits 3,5,7 = 1,0,1 → two 1s → **P1 = 0**
P2 (2,3,6,7)	bits 3,6,7 = 1,1,1 → three 1s → **P2 = 1**
P4 (4,5,6,7)	bits 5,6,7 = 0,1,1 → two 1s → **P4 = 0**
Code word	positions 1…7 → **0 1 1 0 0 1 1**
Error check	If received 0110**1**11 (bit 5 flipped): C1 = 1, C2 = 0, C4 = 1 → C4C2C1 = 101 = 5 → **bit 5 is wrong; flip it**

3 Logic Gates and Universal Gates

A **logic gate** is the basic building block of digital circuits: it takes one or more binary inputs and produces one binary output according to a logical rule.

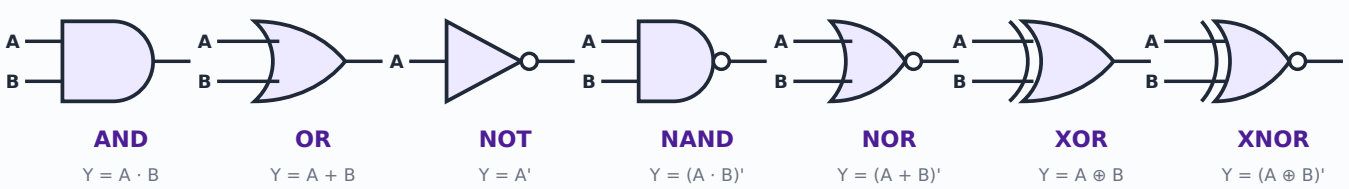


Fig 3.1 Symbols and Boolean expressions of the basic logic gates

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

Gate	Functionality (in words)
NOT (inverter, 1 input: 0→1, 1→0)	Output is the **opposite** of the input
AND	Output is 1 **only when all** inputs are 1
OR	Output is 1 when **at least one** input is 1
NAND / **NOR**	AND / OR followed by NOT (output is the complement)
XOR	Output is 1 when the inputs are **different** (odd number of 1s)
XNOR	Output is 1 when the inputs are the **same** (equality detector)

Universal gates
NAND and **NOR** are called **universal gates** because **any** logic function (NOT, AND, OR and hence all circuits) can be built using only NAND gates or only NOR gates. They are cheap, fast and easy to fabricate.

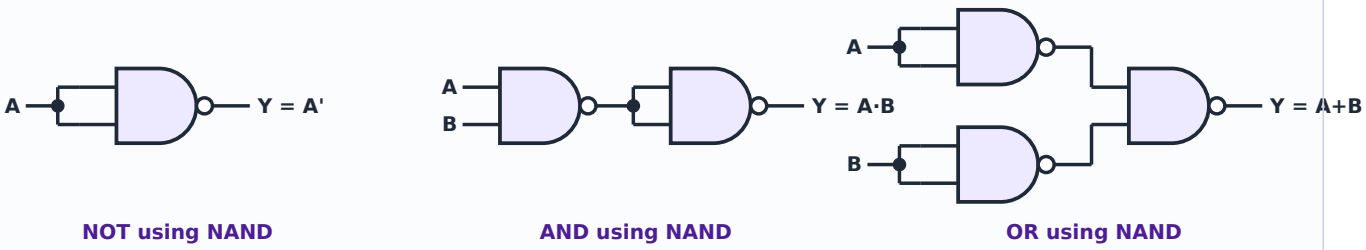


Fig 3.2 NOT, AND and OR gates built using only NAND gates

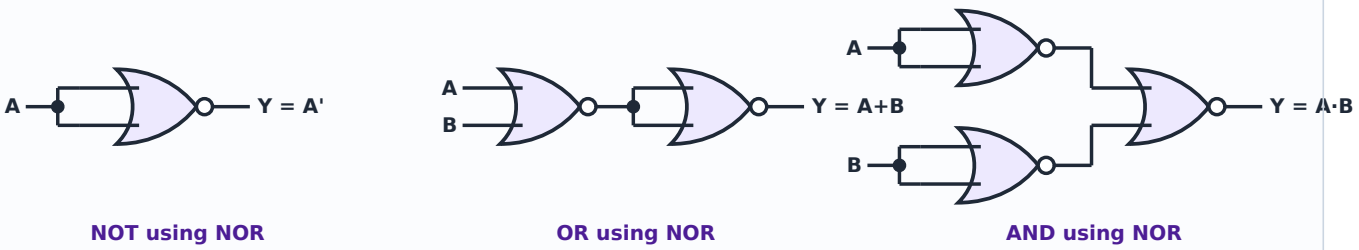


Fig 3.3 NOT, OR and AND gates built using only NOR gates

4 Boolean Algebra: Properties and Theorems

Boolean algebra is the algebra of two-valued variables (0 and 1) with the operations AND (·), OR (+) and NOT (A′ means "NOT A"). It is used to **simplify** logic circuits.

Law	AND form	OR form
Identity	A · 1 = A	A + 0 = A
Null (domination)	A · 0 = 0	A + 1 = 1
Idempotent	A · A = A	A + A = A
Complement	A · A′ = 0	A + A′ = 1
Involution	(A′)′ = A	(A′)′ = A
Commutative	A · B = B · A	A + B = B + A
Associative	(A·B)·C = A·(B·C)	(A+B)+C = A+(B+C)
Distributive	A·(B + C) = AB + AC	A + BC = (A + B)(A + C)
Absorption	A · (A + B) = A	A + A·B = A
Useful	A · (A′ + B) = AB	A + A′B = A + B
De Morgan's	**(A · B)′ = A′ + B′**	**(A + B)′ = A′ · B′**

Duality principle
Every Boolean identity stays true if we swap **AND ↔ OR** and **0 ↔ 1**. (That is why each law above has two forms.)

De Morgan's theorems in words

- The complement of a product = the sum of the complements. • The complement of a sum = the product of the complements.
- In circuits: NAND = OR of inverted inputs; NOR = AND of inverted inputs (bubbled gates).

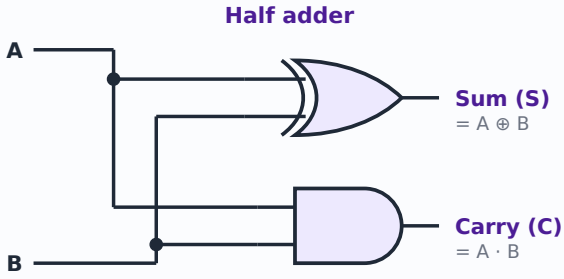
A	B	(A·B)′	A′ + B′	(A + B)′	A′ · B′
0	0	1	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	0	0	0

Simplification example
Simplify Y = A′B + AB′ + AB.
Y = A′B + A(B′ + B) = A′B + A·1 = A′B + A = **A + B** (using A + A′B = A + B). So the three-term circuit is just one OR gate.

5 Simple Combinational Circuits: Half and Full Adder

DEFINITION
A **combinational circuit** has outputs that depend **only on the present inputs** (no memory). Examples: adders, subtractors, multiplexers, decoders, encoders.

Half adder: adds two 1-bit numbers



A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Fig 3.4 Half adder: logic diagram and truth table (Sum = A ⊕ B, Carry = A · B)

Full adder: adds two bits and a carry-in

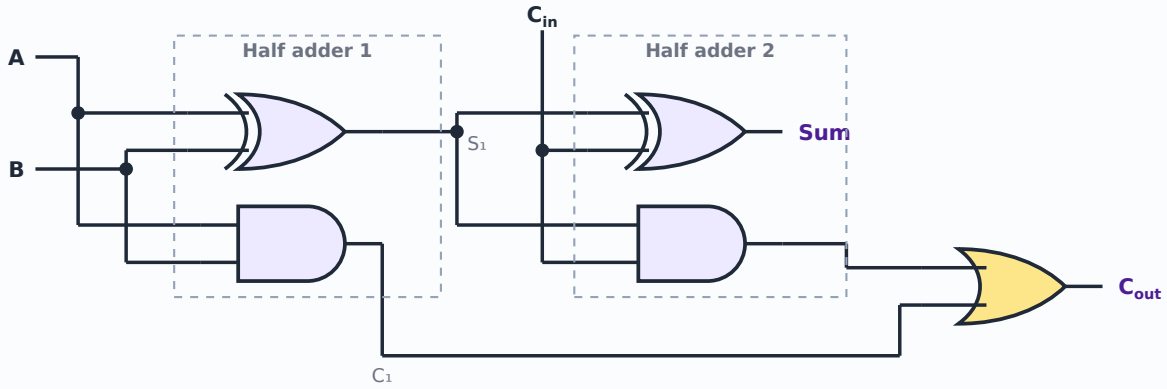


Fig 3.5 Full adder built from two half adders and an OR gate

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Adder equations

Half adder: $S = A \oplus B$, $C = A \cdot B$

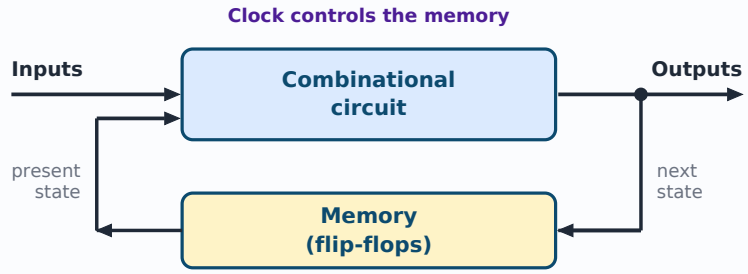
Full adder: $Sum = A \oplus B \oplus C_{in}$, $C_{out} = A{\cdot}B + C_{in}{\cdot}(A \oplus B)$

APPLICATIONS

Arithmetic Logic Unit (ALU) of every processor, calculators, address calculation, counters and digital signal processing. Cascading full adders gives an *n*-bit parallel adder.

6 Introduction to Sequential Circuits and Flip-Flops

	Combinational circuit	Sequential circuit
Output depends on	Present inputs only	Present inputs **and past state**
Memory element	None	Flip-flops / latches
Clock	Not needed	Usually clocked
Examples	Adder, decoder, multiplexer	Registers, counters, memories

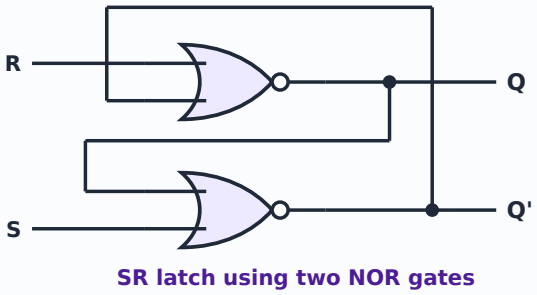


Flip-flop = a 1-bit memory cell with two stable states (Q = 0 or Q = 1). It remembers its state until the inputs and clock change it.

Latch responds to inputs at any time (level sensitive); a **flip-flop** changes only at the clock edge (edge triggered).

A flip-flop has outputs **Q** and its complement **Q′**.

Fig 3.6 Block diagram of a sequential circuit



Working of the NOR latch

S = 1, R = 0 → **Q = 1** (set)

S = 0, R = 1 → **Q = 0** (reset)

S = 0, R = 0 → **holds** the previous state (memory)

S = 1, R = 1 → **forbidden** (both outputs 0)

Fig 3.7 SR latch: the basic memory element

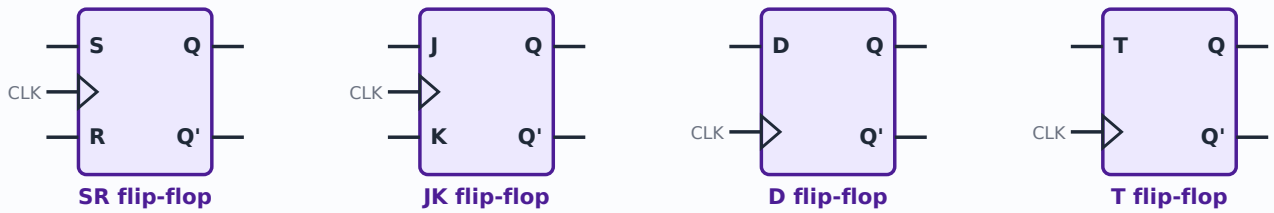


Fig 3.8 Block symbols of SR, JK, D and T flip-flops (triangle = clock input)

S	R	Q^+
0	0	Q
0	1	0
1	0	1
1	1	X

J	K	Q^+
0	0	Q
0	1	0
1	0	1
1	1	Q′

D	Q^+
0	0
1	1

T	Q^+
0	Q
1	Q′

Fig 3.9 Characteristic tables: SR, JK, D and T (Q^+ = next state; X = not allowed)

Flip-flop	Behaviour	Characteristic equation	Remark
SR	Set / Reset / Hold	$Q^+ = S + R'Q$	S = R = 1 is invalid
JK	Like SR, but 11 → **toggle**	$Q^+ = JQ' + K'Q$	Universal flip-flop; race-around in level-triggered form (cured by master-slave / edge triggering)
D	Output follows D after the clock (delay)	$Q^+ = D$	Used in registers and data storage
T	Toggles when T = 1	$Q^+ = T \oplus Q$	JK with J = K = T; used in counters

7 Registers and Counters (Elementary Treatment)

Register
A group of flip-flops that stores a binary word. An *n*-bit register uses *n* flip-flops. A **shift register** also moves the stored bits left or right on every clock pulse.

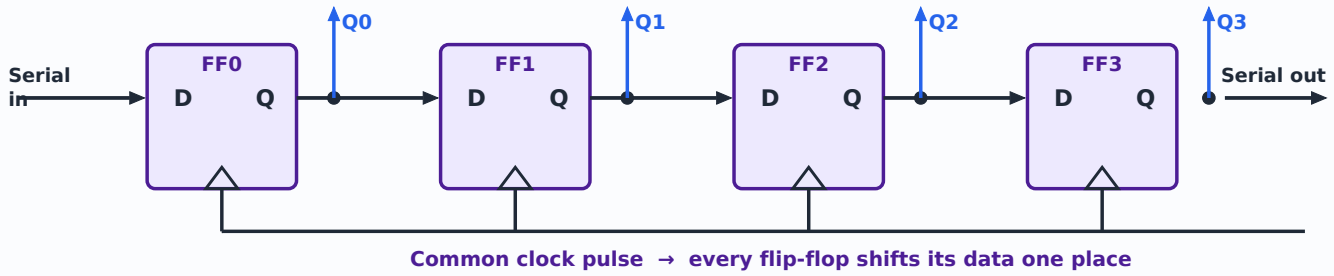


Fig 3.10 4-bit shift register (serial-in, parallel/serial-out) using D flip-flops

Type	Data in	Data out	Typical use
SISO	Serial	Serial	Time delay of digital signals
SIPO	Serial	Parallel	Serial-to-parallel conversion (receiver)
PISO	Parallel	Serial	Parallel-to-serial conversion (transmitter)
PIPO	Parallel	Parallel	Temporary data storage (buffer register)

Counters

DEFINITION
A **counter** is a sequential circuit that goes through a fixed sequence of states when clock pulses arrive. An *n*-bit counter has up to $\mathbf{2^n}$ states (a **mod-2^n** counter).

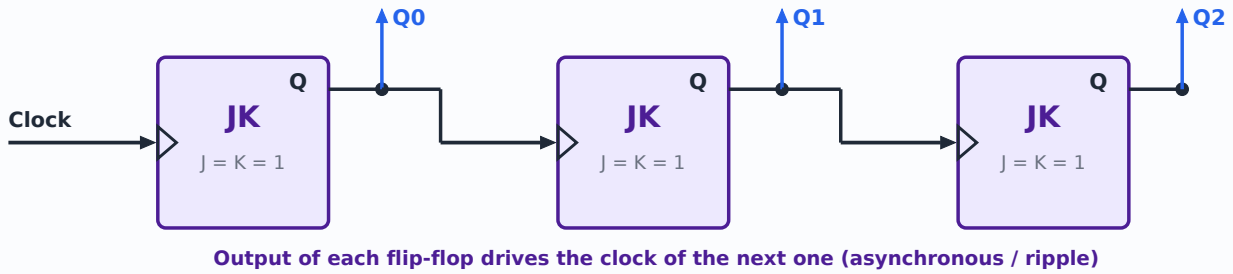
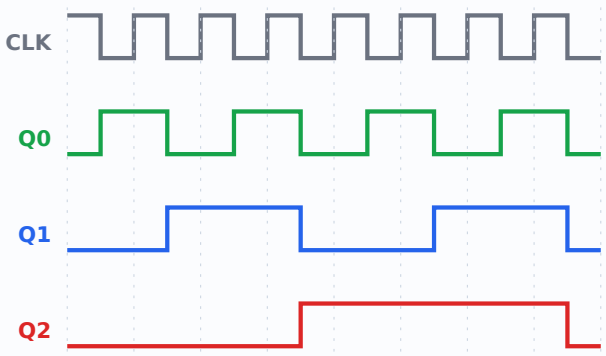


Fig 3.11 3-bit asynchronous (ripple) up-counter using JK flip-flops in toggle mode



Clock	Q2	Q1	Q0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0

Fig 3.12 Timing diagram and count sequence of the 3-bit counter (each stage divides the frequency by 2)

	Asynchronous (ripple) counter	Synchronous counter
Clock	Clock drives only the first FF; each FF clocks the next	**Same clock** to all flip-flops
Speed	Slower (delays add up)	Faster
Circuit	Simple, fewer gates	Needs extra gates; more complex

- **Up / down counters** count 0→7 or 7→0 (down counter takes the clock from Q′). **Mod-N counter:** resets after N states, e.g. a **decade (mod-10) counter** uses 4 flip-flops and clears at 1010.
- **Applications:** digital clocks and timers, frequency dividers, event counting, address generation in memories.

8 Quick Summary and Exam Corner

Unit III in one glance

- Binary (base 2), octal (8), hex (16). Group bits in 3s for octal and 4s for hex. 2’s complement = 1’s complement + 1.
- BCD = digit-by-digit 8421; Excess-3 = BCD + 3; Gray = one-bit change; Hamming (7,4) = 3 parity bits at 1, 2, 4 correct a single error.
- Gates: AND, OR, NOT, NAND, NOR, XOR, XNOR; NAND and NOR are universal. De Morgan: $(AB)' = A'+B'$, $(A+B)' = A'B'$.
- Half adder: $S = A \oplus B$, $C = AB$. Full adder: $S = A \oplus B \oplus Cin$, $Cout = AB + Cin(A \oplus B)$.
- Sequential = combinational + memory. Flip-flops: SR, JK (toggle at 11), D ($Q^+ = D$), T (toggle).
- Shift registers: SISO, SIPO, PISO, PIPO. n flip-flops give a mod-2^n counter; ripple = asynchronous, others = synchronous.

Short answer questions (2 marks)

1. Convert $(45)_{10}$ to binary, and $(1011\ 1100)_2$ to hexadecimal.
2. What is a universal gate? Name two.
3. State De Morgan’s theorems.
4. Why is Gray code used in position encoders?
5. Write the Sum and Carry expressions of a half adder.
6. Differentiate combinational and sequential circuits.
7. What is a flip-flop? How many states does it have?
8. What is the use of a shift register?

Long answer questions (5 / 10 marks)

1. Explain BCD, Excess-3 and Gray codes with a table for the digits 0 to 9.
2. Explain the Hamming code. Encode 1011 using even parity and show how a single-bit error is corrected.
3. Draw the symbol, truth table and expression of all basic gates (NOT, OR, AND, NOR, NAND, XOR, XNOR).

4. Show that NAND and NOR gates are universal by realising NOT, AND and OR gates using each.
5. State and prove the basic theorems of Boolean algebra, including De Morgan's theorems, using truth tables.
6. Design a half adder and a full adder; give truth tables and logic diagrams.
7. Explain the SR, JK, D and T flip-flops with symbols and characteristic tables.
8. Explain a 4-bit shift register and a 3-bit ripple counter with diagrams and timing.

Numerical practice (with answers)

1. Convert $(110101)_2$ to decimal, octal and hex. *Ans: 53, 65_8, 35_{16}*
2. Find the 2's complement of 01100. *Ans: 10011 + 1 = 10100*
3. Convert binary 1100 to Gray code. *Ans: 1010*
4. Simplify Y = AB + AB′. *Ans: Y = A(B + B′) = A*
5. How many flip-flops are needed for a mod-12 counter? *Ans: 4 (since $2^3 = 8 < 12 \le 2^4 = 16$)*

Quick check: pick the right answer

1. The output of a 2-input XOR gate is 1 when the inputs are: (a) same (b) different (c) both 1. *Ans: (b)*
2. Which flip-flop toggles for J = K = 1? (a) SR (b) D (c) JK. *Ans: (c)*
3. A 3-bit counter has how many states? (a) 3 (b) 6 (c) 8. *Ans: (c)*
4. Excess-3 code of decimal 4 is: (a) 0100 (b) 0111 (c) 1000. *Ans: (b)*